ENGINEERING REPORT

RecoverChain: architecture and delivered capabilities

RecoverChain engineering · internal snapshot

2026-08-31 · branch master @ 6fff8c3 · 23 commits · 205 backend tests passing

ABSTRACT

RecoverChain is a revenue-recovery decisioning engine. Failure events — failed payments, overdue invoices, abandoned checkouts — are correlated into *cases*, and every case runs one fixed deterministic lifecycle: assess risk, diagnose cause, predict recovery, recommend an action with an expected recoverable value, pass a policy and safety gate, execute against an adapter, verify independently against settlement, and audit. A second subsystem, the Dataset Lab, ingests tabular files, maps columns to canonical fields, checks machine-learning readiness, trains per-dataset models, and replays rows back through the pipeline.

This report describes what has been built: a ten-step guided workflow paired with a gated Insights report; a policy engine that is the sole execution authority, backed by a real attempt ledger; machine learning that trains on and scores real uploaded data as advisory telemetry; a fully offline-capable stack in Indian rupees; and a case-generation path whose predictor is now loaded once per request rather than once per row, cutting initialisations from 78 to 1 and endpoint time from 3.23 s to 2.65 s on the reference run.

1 System overview

Failure events are correlated into **cases**. Each case runs the fixed lifecycle end to end — there is no branching and no discretion outside the policy gate:

```
ingest → assess-risk → diagnose → predict-recovery → recommend-action (+ERV)
       → policy / safety gate → agent execution → verify → audit
```

The Dataset Lab is the second subsystem. It ingests CSV, Parquet, and XLSX; maps columns to canonical fields; runs quality and leakage checks against a minimum-information contract; trains per-dataset models; and replays dataset rows through the same pipeline to produce real cases.

The product surface is two "models". The first, **RecoverChain** (route `/`), is a ten-step guided workflow in which one step fills the screen at a time, navigation is a single `Proceed →`, and every screen is explicit about how each figure was produced. The second, **Insights** (route `/insights`), is a plain-language report aggregating every recovery run; it unlocks once the workflow has been completed.

2 Stack

- **Backend.** FastAPI, SQLAlchemy 2.0, Alembic, Pydantic v2, `Decimal` money. SQLite is the offline default; Postgres is supported, with its schema owned by migrations.
- **Frontend.** React 18, Vite 5, react-router v6, Tailwind v3, with a lazy-loaded Three.js / React-Three-Fiber / `@react-spring/three` / GSAP motion backdrop that is toggleable and `prefers-reduced-motion`-safe.
- **Machine learning.** scikit-learn and XGBoost, running as advisory telemetry.
- **Offline-first.** No CDN, no runtime web fonts, Redis optional, all dependencies bundled. The application runs with no network.

3 Architecture

The backend is cleanly layered.

3.1 domain/

Pydantic models: the `RecoveryCase` aggregate and seven stage sub-objects, `CaseState`, and `Money` (a `Decimal`). Also `lifecycle.py`, `interfaces.py`, `llm_schemas.py`.

3.2 application/

One engine per stage: `risk_detector`, `diagnosis_engine`, `recovery_predictor` (deterministic baseline) and `recovery_predictor_ml` (per-dataset model), `action_evaluator`, `policy_engine`, `agents`, `verification_engine`.

`case_pipeline.py` (`CasePipelineService`) runs assess, diagnose, predict, recommend, and policy in one call and is the shared path for `/events/batch`, `/cases/{id}/advance`, and `/datasets/{id}/generate-cases`. The Dataset Lab lives here too: `dataset_lab`, `dataset_intelligence`, `ml_readiness`, `ml_training`.

3.3 infrastructure/

`orm.py` and `dataset_orm.py`: lifecycle sub-objects are stored as JSON columns on `recovery_cases`, alongside the `execution_attempts` ledger and `idempotency_keys`. `repositories.py` maps ORM to domain and back and provides `record_execution_attempt` and `get_policy_context`. Also `redis_client.py` and `adapters.py`.

3.4 api/

`auth.py` (`verify_api_key`); `main.py` (the case pipeline, `/events/batch`, `/cases/{id}/advance` and `/stop`, request-id middleware); `dataset_router.py` (`/datasets/*`); `system_router.py` (`/system/*`, read-only observability).

4 The guided workflow

The workflow was consolidated from eleven steps to ten by merging the decision and policy screens. Each step is driven entirely by live data.

Table 1. *The ten workflow steps and their data sources.*

#	Step	What it shows
01	Upload data	Dropzone or bundled sample; the formats the backend accepts.
02	Data review	Real filename, record count, columns, detected canonical fields.
03	Data quality	`data_quality_report` score and breakdown, completeness, leakage check.
04	Data mapping	The four roles — Customer, Amount, Date, Result — pre-filled from detection; low-confidence guesses flagged.
05	Data connection	How the file is wired into the pipeline: canonical inputs, model features versus held-out columns with reasons, split strategy, class balance, readiness verdict.
06	Revenue risk detection	Runs `generate-cases`; shows real counters for rows accepted and skipped, with reasons.
07	AI analysis	The highest-value case: risk cause and confidence, recovery probability, expected recoverable value. Rule-based and model contributions are labelled.
08	Decision & policy	Merged. The rules-engine proposal and the policy engine's rule-by-rule verdict on one screen.
09	Recovery	If permitted, executed; if escalated, an explicit approve-and-execute; if blocked, the reason.
10	Verified result	Outcome verified independently against the settlement source; actual amount recovered.

> Reaching step 10 sets a `rc-workflow-done` flag in local storage, which unlocks the Insights report.

5 Design invariants

These properties are enforced in code and covered by tests.

- **The policy engine is the sole execution authority.** `DeterministicPolicyEngine` decides; no model value feeds a `PolicyDecision`. When no per-dataset model applies, prediction falls back to `DeterministicBaselinePredictor` — a real estimate, never a silent `0.0`.
- **Execution is gated.** `AgentOrchestrator.execute` requires a persisted `PolicyDecision.status == PERMITTED`, a matching action, a fresh decision, and a capable agent.
- **Policy uses real history.** `PolicyContext` is built from the `execution_attempts` ledger, so retry, cooldown, and contact limits count real attempts. It also enforces `StopRule` and `ConsentCheck`.

- **Financials are exact.** `Money` is a `Decimal`; the ORM column is `Numeric(18,4)`; the default currency is INR. The recovered amount comes only from `RecoveryOutcome`, clamped to `[0, amount]`.
- **Leakage and dataset–model isolation.** `DatasetValidator.detect_leakage`, the minimum-information contract, and a predictor that loads only a model whose metadata `dataset_id` matches exactly.
- **Migrations own the Postgres schema.** `create_all` runs only for SQLite.
- **Authentication.** `verify_api_key` guards every mutating route.

6 Capabilities

Table 2. *What the system does today.*

Area	Delivered
Case correlation, lifecycle, state machine	yes
Risk, diagnosis, recommendation, ERV engines	yes — deterministic rules
Policy engine, safety gate, ledger-backed context	yes
Independent settlement verification	yes
Audit trail (`audit_records`)	yes
Dataset ingest, mapping, quality and leakage checks, readiness	yes
Per-dataset model training and scoring on real data	yes — advisory telemetry
Ten-step guided workflow	yes
Gated Insights aggregation report	yes
Fully offline operation, Indian rupees	yes
Docker Compose, CI, request-id logging	yes

7 Machine learning

The machine-learning subsystem trains on real uploaded data and runs as advisory telemetry.

- **Readiness gate** (`ml_readiness.py`). Derives the target, the feature columns, and the excluded columns — identifier, constant, post-outcome leak, high cardinality — along with the temporal split strategy and class balance from the mapped dataset.
- **Training** (`ml_training.py`). A scikit-learn `Pipeline`. A `_coerce_frame` step turns number-like and date-like text columns into numeric and epoch values *at fit and predict time*, so a real file transforms identically when

served. Then impute, scale, and one-hot with `handle_unknown="ignore"`; logistic-regression and XGBoost candidates, calibrated on the validation fold; a chronological split on the real date column, with a stratified fallback — flagged in metadata — when there is no usable date.

- **Quality gate.** ROC-AUC ≥ 0.55 always applies, falling back to validation ROC when the test fold is single-class. `ML_ADAPTIVE_GATE` (default on) scales the row floor down to 15 for smaller real datasets. A model that clears the bar is served; one that does not stays registered for inspection while the pipeline uses the deterministic baseline.
- **Auto-train.** `generate-cases` trains a per-dataset model — rows ≥ 20, readiness ready, a strict 0.55 bar — before replaying rows, and feeds the predictor the actual source row and real timestamp.
- **Serving.** Recovery probability is `1 - failure_risk`; it feeds the ERV and ranking.

> Observed: a 140-row engineered dataset auto-trains an XGBoost model (ROC ≈ 1.0) that then scores its cases from the trained model, while noisier small datasets are served by the deterministic baseline.

8 Development history

Twenty-three commits, in three phases.

8.1 Foundations

`87e0464`	baseline after fixing the dependency manifests
`3c48888`	Alembic builds the schema from an empty database
`a2a9aec`	isolated pytest database; fixed a `/system/policy` crash
`8c0a4a5`	made the live case loop flow end to end
`1da686f`	ledger-backed policy, real idempotency, honest verification
`be94ade`	ML training quality gate and model card
`283f079`	auth on the pipeline, request logging, N+1 fix, Docker, CI

8.2 Product and UX

`6ec8cd0`	real styling and a corrected case-detail data contract
`5dbc74e`	remove every runtime external dependency — fully offline
`25aad21`	fix dataset upload for repeated filenames
`8431066`	restructure the frontend as one plain-language flow; make the wizard work

`d581a5b` redesign as a single-step guided workflow

`19569c2` 3D and GSAP motion layer

`9d93e82` two models — guided workflow and a gated Insights report

`1cfc922` reconstruct Step 05 into a real pipeline-wiring view

8.3 This iteration

`c87c158` rupee (INR) currency throughout — `Money`, ORM, schema, and pipeline defaults, plus `en-IN` grouping and ₹ on the frontend

`c87c158` merge the Recovery-decision and Policy-check steps into one “Decision & policy” step; relabel “Send to a person to decide” as “Needs human review”

`c87c158` machine learning that trains on and scores real uploaded data: text and date coercion in the served pipeline, a stratified split fallback, an adaptive quality gate, auto-train in `generate-cases`, real row features to the model

`6fff8c3` build the model predictor once per `generate-cases` request; give Step 06 an honest processing state

9 Performance improvement

Case generation constructed a fresh predictor on every row of `generate-cases`, each construction scanning the model registry and loading the pipeline from disk.

`predict_recovery_for_case(case, dataset_id=None, predictor=None)` now takes an optional pre-built predictor; `CasePipelineService.predict` and `.advance` thread it through; `generate_cases` builds **one** predictor and reuses it for every row. The public API is unchanged — all new parameters are optional. Step 06 now shows an honest `“Generating recovery cases…”` processing state.

Table 3. *Before and after, on a 77-case run against the real 380-model registry.*

	Predictor initialisations	Endpoint time
Before	78	3.23 s
After	1	2.65 s

The per-row model load — roughly 130 ms each when a model exists — is now paid once, and the saving scales with row count and registry size. A new regression test, `test_generate_cases_predictor_reuse.py`, asserts one predictor construction per request and no duplicate cases.

10 Current state

- **Backend.** 205 tests pass. Run from `backend/` with `PYTHONPATH=.` on `backend/venv`.
- **Frontend.** `npm run build` is clean: the main bundle is about 307 kB (102 kB gzipped); the Three.js `Backdrop` chunk is about 858 kB (236 kB gzipped) and is lazy-loaded.
- **Demo data** (`backend/demo.db`). 33 datasets, roughly 684 cases, INR throughout, with trained models wherever a signal clears ROC 0.55.
- **System health.** `ml_subsystem: SHADOW_ONLY / ADVISORY_TELEMETRY_ONLY`; `policy_engine: DETERMINISTIC_AUTHORITY / gate ENFORCED`; `execution_engine: MockExecutionAdapter`.

Appendix A. Commands and configuration

```
# full stack — backend :8000 + frontend :5173 (SQLite)
python run.py

# backend
cd backend && PYTHONPATH=. uvicorn api.main:app --reload
cd backend && PYTHONPATH=. pytest                # 205 tests
cd backend && PYTHONPATH=. alembic upgrade head   # Postgres only

# frontend
cd frontend && npm install && npm run dev
cd frontend && npm run build

# containers
docker-compose up --build
```

Table 4. *Environment variables.*

Variable	Default	Notes
`DATABASE_URL`	`postgresql://…`	`sqlite:///./x.db` for local
`API_KEY`	`test-api-key`	`X-API-Key` on mutating routes
`ML_MIN_ROC_AUC` / `ML_MIN_TEST_ROWS`	`0.55` / `200`	training quality gate; `0` disables
`ML_ADAPTIVE_GATE`	`1`	scale the row floor down for small datasets
`REDIS_URL`	`redis://localhost:6379/0`	only `/health` uses it; optional